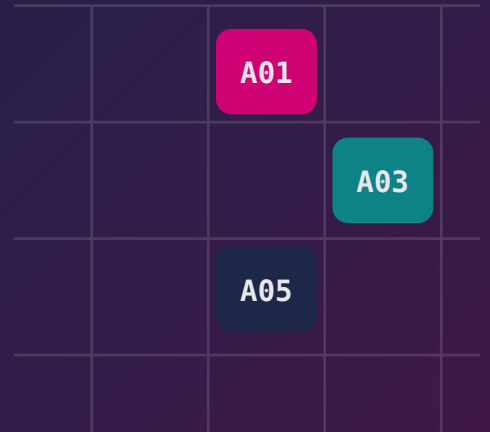


Techniques défensives et offensives des applications Web · Module 5

Les vulnérabilités du développement — partie 1

OWASP Top 10:2025, A01 → A05. Pour chaque catégorie : mécanisme, **code vulnérable** réaliste, correctif de **niveau production**, impact et CWE.



Objectifs pédagogiques

Public : développeurs, architectes, pentesters, RSSI. À l'issue :

- Reconnaître le **code vulnérable** dans ses propres bases.
- Écrire le **correctif robuste** (pattern, API, configuration).
- Connaître les **pièges** que les correctifs « évidents » ne couvrent pas.
- Citer les **CWE** pour tracer et prioriser.

La grille, pour chaque catégorie

① **mécanisme** → ② **code vulnérable** → ③ **correctif de prod** → ④ **impact et CWE**. On traite **plusieurs facettes** par catégorie (ex. A01 : BOLA/IDOR, accès fonctionnel, mass assignment).

Parti pris

Pas de snippet « jouet » : le code montre les **vraies API** (Spring Security, JPA, Argon2, Bucket4j...) et les **subtilités** attendues d'un pro.

Le Top 10 OWASP — édition 2025

A01

— Broken Access Control
(absorbe SSRF)

A02

— Security Misconfiguration
(était A05)

A03

— Software Supply Chain Failures
(nouveau)

A04

— Cryptographic Failures
(était A02)

A05

— Injection
(était A03)

A06

— Insecure Design

A07

— Authentication Failures

A08

— Software or Data Integrity Failures

A09

— Logging & Alerting Failures

A10

— Mishandling of Exceptional Conditions
(nouveau)

Ce module

Les 5 premières catégories (A01–A05). A06–A10 sont traitées en partie 2. **Changement 2025** : Security Misconfiguration monte en A02, Supply Chain entre en A03, Injection et Crypto reculent, Insecure Design passe en A06.

OWASP Top 10 et Top 25 CWE

OWASP Top 10

Des **catégories de risques** (A01...A10) — vue stratégique, orientée cause.

Top 25 CWE / MITRE

Une liste des **faiblesses logicielles** précises (**CWE** = Common Weakness Enumeration) — vue technique, traçable.

Elles s'emboîtent

Une catégorie OWASP **regroupe plusieurs CWE**. L'**Injection (A05:2025)** rassemble **CWE-89** (SQLi), **CWE-79** (XSS), **CWE-78** (commandes)... Citer la CWE rend un *finding* **précis, priorisable et vérifiable**.

Rappel module 2

Le CWE Top 25 2025 place XSS, SQLi et CSRF en tête : les faiblesses Web dominant. OWASP donne la stratégie, la CWE la traçabilité.

A01

Broken Access Control

Agir hors de ses permissions : sur des objets, des fonctions ou des champs. **Toujours n°1.**

Dans cette catégorie

- Trois formes : BOLA/IDOR, BFLA, mass assignment
- Scoper la requête
- Sécuriser chaque fonction
- DTO en liste blanche
- Impact et CWE

Trois formes à connaître



BOLA / IDOR

Accéder à l'objet d'**autrui** en changeant un identifiant (`/facture/1043`). Contrôle au niveau **objet**. (2025 : BOLA explicite.)



BFLA — accès fonctionnel

Atteindre une **fonction** réservée (endpoint admin) par *forced browsing*. Contrôle au niveau **fonction**.



Mass assignment

Modifier un **champ** non prévu (`role=ADMIN`) via le binding automatique. Contrôle au niveau **attribut**.

Le principe directeur

Le contrôle d'accès se décide **côté serveur**, à **chaque** requête, et par **refus par défaut**. L'interface (bouton caché) ne protège rien.

Nouveauté 2025 : la SSRF est désormais rattachée à cette catégorie.

Scoper la requête, ne pas « charger puis vérifier »

✗ Vulnérable

```
@GetMapping("/api/factures/{id}")
Facture get(@PathVariable Long id) {
    // aucun lien avec l'utilisateur courant
    return repo.findById(id).orElseThrow();
}
```

Même « findById puis if owner » est fragile : on l'oublie une fois, la faille revient.

Réflexe pro

Retourner **404** (ne pas confirmer l'existence). Envisager des identifiants **non devinables** (UUID) en défense en profondeur, et une **row-level security** côté base pour les cas critiques.

✓ Correctif — la requête porte l'autorisation

```
@GetMapping("/api/factures/{id}")
Facture get(@PathVariable Long id,
            @AuthenticationPrincipal User u) {
    // on ne CHARGE jamais l'objet d'un autre
    return repo.findByIdAndOwnerId(id, u.id())
        .orElseThrow(NotFound::new); // 404, pas 403
}
// Repo : SELECT ... WHERE id=? AND owner_id=?
```

Sécuriser chaque fonction, par défaut

✗ Vulnérable — endpoint admin « oublié »

```
// aucun contrôle : accessible par forced browsing
@PostMapping("/api/admin/users/{id}/promote")
void promote(@PathVariable Long id) {
    users.setRole(id, Role.ADMIN);
}
```

✓ Correctif — deny-by-default + method security

```
// 1) tout est refusé sauf ce qui est autorisé
http.authorizeHttpRequests(a -> a
    .requestMatchers("/api/admin/**").hasRole("ADMIN")
    .anyRequest().authenticated());
// 2) défense en profondeur au niveau méthode
@PreAuthorize("hasRole('ADMIN')")
void promote(Long id) { ... }
```

Piège classique : sécuriser l'UI mais laisser l'API ouverte. Le contrôle doit être sur le **endpoint**, pas sur le lien qui y mène. **Refus par défaut** pour qu'un nouvel endpoint soit protégé même si on oublie l'annotation.

Ne jamais binder l'entité directement

✗ Vulnérable — le client fixe ses propres droits

```
@PostMapping("/api/profil")
void update(@RequestBody User body) {
    // body = {"name":"x","role":"ADMIN","balance":9999}
    users.save(body); // tous les champs écrasés !
}
```

✓ Correctif — DTO en liste blanche

```
// seuls les champs autorisés existent dans le DTO
record ProfilUpdate(String name, String email) {}

@PostMapping("/api/profil")
void update(@Valid @RequestBody ProfilUpdate in,
            @AuthenticationPrincipal User u) {
    users.updateProfile(u.id(), in.name(), in.email());
}
// role, balance... ne sont JAMAIS mappés depuis l'entrée
```

Règle

Séparer le **modèle d'entrée** (DTO) du **modèle de persistance** (entité). Ne jamais désérialiser directement dans l'entité ; annoter les champs sensibles `@JsonIgnore` en dernier recours.

Conséquences et contre-mesures

Impact

- Fuite / modification de données d'autrui
- Élévation de privilège (→ admin)
- Prise de contrôle métier (fraude)
- SSRF : accès aux ressources internes (nouveau 2025)

CWE-284 Access Control **CWE-639** BOLA/IDOR **CWE-862** Missing Authorization **CWE-915** Mass Assignment **CWE-918** SSRF

Contre-mesures

- **Refus par défaut**, autorisation **centralisée**
- Scoper les requêtes au sujet, contrôler la **propriété**
- **DTO** en entrée, jamais l'entité
- Pour la SSRF : **allowlist** des destinations sortantes, blocage des métadonnées cloud
- Tests d'accès automatisés (chaque rôle × chaque endpoint)

A02

Security Misconfiguration

Le code est sûr, mais la configuration ouvre la porte. **Monté de A05 (2021) à A02 en 2025.**

Dans cette catégorie

- Réglages par défaut, surface, verbosité
- Une configuration de production
- En-têtes de sécurité et CORS
- Impact et CWE

Réglages par défaut, surface inutile, verbosité

Définition

Tous les réglages **par défaut**, **trop permissifs** ou **oubliés** qui exposent l'application — souvent **sans une ligne de code fautive**.

Pourquoi #2 en 2025

Les erreurs de configuration explosent avec le cloud, les conteneurs et l'IaC : un **template fautif se réplique à grande échelle**. Exploiter une misconfig est souvent plus simple qu'un zero-day.

Cas typiques

- Comptes / mots de passe **par défaut** (admin/admin)
- Erreurs **verbeuses** (stack traces exposées)
- Consoles / actuator / debug **exposés**
- **En-têtes de sécurité** absents, **CORS** permissif
- Buckets, rôles IAM, WAF mal réglés

Une configuration de production minimale

✗ Vulnérable (application.properties)

```
server.error.include-stacktrace=always
spring.h2.console.enabled=true
management.endpoints.web.exposure.include=*
# comptes de démo jamais retirés
```

✓ Corrigé

```
server.error.include-stacktrace=never
server.error.include-message=never
spring.h2.console.enabled=false
management.endpoints.web.exposure.include=health
management.endpoint.health.show-details=never
```

Principe

Config **minimale**, **reproductible** (Infrastructure as Code), **durcie par défaut**. Séparer les profils **dev** / **prod**. Retirer tout ce qui n'est pas nécessaire en production ; scanner la config (Trivy, kube-bench, Checkov).

Poser les protections navigateur, correctement

✓ En-têtes de sécurité (Spring Security 6)

```
http.headers(h -> h
    .contentSecurityPolicy(c -> c.policyDirectives(
        "default-src 'self'; object-src 'none'"))
    .httpStrictTransportSecurity(s -> s
        .maxAgeInSeconds(31536000).includeSubDomains(true))
    .frameOptions(f -> f.deny()) // anti-clickjacking
    .contentTypeOptions(Customizer.withDefaults()));
```

✗ CORS grand ouvert

```
cfg.setAllowedOrigins(List.of("*"));
cfg.setAllowCredentials(true); // combinaison interdite
```

✓ CORS restreint

```
cfg.setAllowedOrigins(List.of(
    "https://app.mondomaine.fr")); // origine explicite
cfg.setAllowedMethods(List.of("GET", "POST"));
```

Conséquences et contre-mesures

Impact

- Accès via un compte / service par défaut
- Fuite d'informations techniques (erreurs, dumps)
- Point d'entrée via un endpoint de debug / CORS laxiste

CWE-16 Configuration **CWE-611** XXE **CWE-756** Missing Error Page

CWE-942 Permissive CORS

Contre-mesures

- **Durcissement** reproductible (laC), profils séparés
- Retirer comptes / services / pages par défaut
- Erreurs **génériques**, en-têtes de sécurité, CORS restreint
- Scanner la configuration en continu (module 3)

A03

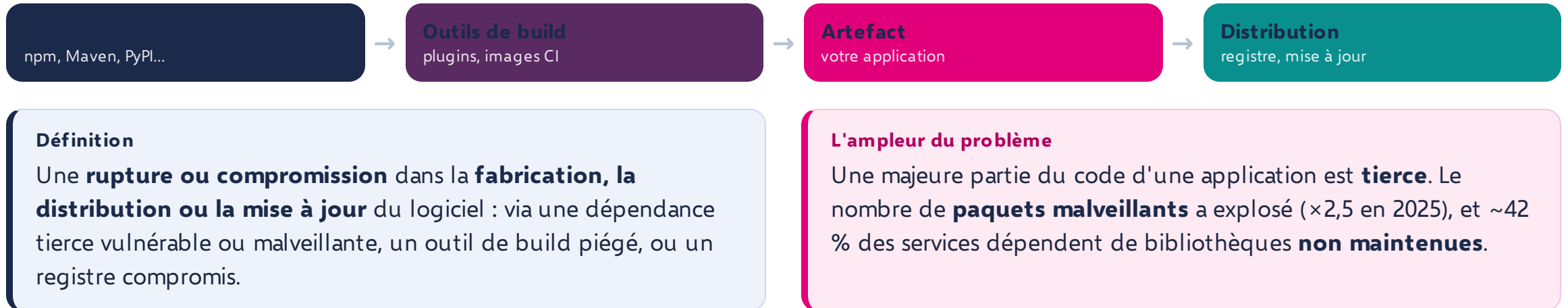
Software Supply Chain Failures

Nouvelle catégorie 2025. Le risque ne vient plus de votre code, mais de tout ce dont il dépend.

Dans cette catégorie

- De « composants vulnérables » à toute la chaîne
- Verrouiller et vérifier les dépendances
- Scanner en continu (SCA + SBOM)
- Durcir le pipeline
- Impact et CWE

Toute la chaîne de fabrication est une cible



Versions figées, intégrité vérifiée

X Vulnérable — plages ouvertes, aucune vérification

```
// package.json – accepte n'importe quelle 4.x
"dependencies": {
  "express": "^4.0.0", // résolu au hasard du build
  "lodash": "*"        // n'importe quelle version
}
// pas de lockfile committé → builds non reproductibles
```

✓ Correctif — lockfile + intégrité

```
# installation reproductible, exacte au lockfile
npm ci          # pas 'npm install'
# package-lock.json fige version ET hash d'intégrité :
"lodash": {
  "version": "4.17.21",
  "integrity": "sha512-v2k...==" # vérifié
}
```

Principe

Committier le **lockfile**, installer en mode **reproductible** (`npm ci`, `mvn -o` avec versions figées). Le **hash d'intégrité** garantit qu'on installe bien le paquet attendu, pas un substitut malveillant.

SCA et SBOM dans le pipeline

✓ Analyse des dépendances (CI)

```
# échoue le build si CVE HIGH/CRITICAL
trivy fs --exit-code 1 \
  --severity HIGH,CRITICAL .

# générer le SBOM (inventaire des composants)
trivy fs --format cyclonedx -o sbom.json .

# rescanner le SBOM à chaque nouvelle CVE,
# sans reconstruire l'application
```

Rappel module 3

Le **SBOM** (Software Bill of Materials) est l'inventaire des composants : il répond en secondes à « suis-je affecté par cette CVE ? ».

Défense en profondeur

- **SCA** à chaque commit (Trivy, Dependency-Check, Gype)
- **Renovate / Dependabot** pour les mises à jour
- Scanner aussi les **images de base** Docker
- Registre de dépendances **filtré** (curation à l'ingestion)

Provenance et intégrité des artefacts

Le pipeline est une cible

Un pipeline compromis signe et distribue du code malveillant avec **votre** confiance (cf. incidents SolarWinds, xz). Il faut prouver **d'où vient** l'artefact et qu'il n'a pas été altéré.

Bonnes pratiques

- **Signer** les artefacts et images (Sigstore / cosign)
- Générer une **provenance** vérifiable (cadre **SLSA**)
- **Figurer** les actions/plugins CI par hash de commit
- Secrets du pipeline en **coffre**, accès minimal

```
# signer une image (Sigstore / cosign)
cosign sign registry/monapp@sha256:abc...

# vérifier la signature avant déploiement
cosign verify registry/monapp@sha256:abc... \
  --certificate-identity ci@monorg.fr

# figer une action GitHub par hash (pas par tag)
uses: actions/checkout@8f4b7f8... # pas @v4
```

Conséquences et contre-mesures

Impact

- Exécution de code via une dépendance piégée
- Compromission massive (tous les utilisateurs de l'artefact)
- Vol de secrets du pipeline
- Angle mort : une CVE sans signature détectable

CWE-1104 Composant non maintenu **CWE-1035** Composant vulnérable

CWE-506 Code malveillant **CWE-829** Inclusion non fiable

Contre-mesures

- **Lockfiles** + installation reproductible + hash d'intégrité
- **SCA** en CI + **SBOM** tenu à jour
- Mises à jour automatisées (Renovate / Dependabot)
- **Signature** et **provenance** (cosign, SLSA)
- Pipeline durci, dépendances CI figées

A04

Cryptographic Failures

Données sensibles mal protégées : au stockage, en transit, dans la conception cryptographique.

Dans cette catégorie

- Trois fronts
- Mots de passe : hachage lent
- Données au repos : AES-GCM + KMS
- Données en transit : TLS
- Impact et CWE

Trois fronts



Mots de passe

Stockés en clair, en MD5/SHA-1, ou hachés **trop vite** → cassables massivement.



Données au repos

Chiffrement absent, mode **ECB**, non authentifié, ou clé **en dur** dans le code.



Données en transit

HTTP en clair, TLS obsolète, absence de **HSTS** → interception.

Fil rouge (rappel module 4)

On ne **chiffre** pas un mot de passe (réversible), on le **hache**. Et on n'invente **jamais** sa propre cryptographie — on utilise des primitives éprouvées, correctement paramétrées. Penser aussi à la préparation **post-quantique** pour les données à longue durée de vie.

Hachage lent, salé, paramétré

X Vulnérable

```
// MD5 non salé : rainbow tables, GPU
user.setPassword(md5(pwd));
// ou SHA-256 « simple » : trop rapide

// vérification non constante :
if (stored.equals(hash(input))) ... // timing
```

✓ Correctif — Argon2id calibré

```
// sel, longueur, parallélisme, mémoire(Ko), itérations
var enc = new Argon2PasswordEncoder(
    16,32,1,1<<16,3); // 64 Mo, 3 iters
String h = enc.encode(pwd); // sel aléatoire inclus
boolean ok = enc.matches(pwd, h); // temps constant
// re-hacher au login si params obsolètes (upgrade)
```

À calibrer

Régler les paramètres pour viser **~0,5–1 s** par hash sur votre matériel. Alternatives acceptables : **bcrypt** (coût ≥ 12), **scrypt**. Ajouter un **poivre** (secret côté serveur, hors base) est un plus.

Chiffrement authentifié + gestion des clés

X Vulnérable

```
// ECB (motifs visibles) + clé en dur
Cipher c = Cipher.getInstance("AES/ECB/PKCS5");
c.init(ENCRYPT, new SecretKeySpec(
    "1234567890123456".getBytes(), "AES"));
```

✓ Correctif — AES-256-GCM + KMS

```
// GCM = confidentialité + intégrité (authentifié)
Cipher c = Cipher.getInstance("AES/GCM/NoPadding");
byte[] iv = new byte[12]; rng.nextBytes(iv); // unique !
c.init(ENCRYPT, dataKey, new GCMParameterSpec(128, iv));
byte[] ct = c.doFinal(clair); // stocker iv || ct
// dataKey chiffrée par un KMS (envelope), rotée
```

Niveau pro

IV unique par message (jamais réutilisé en GCM). **Envelope encryption** : le KMS chiffre la clé de données, la clé maître ne quitte jamais le KMS. Prévoir **rotation** et **agilité**. Préférer une lib de haut niveau (**Google Tink**, libsodium) au JCA brut.

TLS partout, et les conséquences

En transit

- **TLS 1.2 min** (idéalement 1.3), suites fortes, forward secrecy
- **HSTS** : `max-age=31536000; includeSubDomains`
- Rediriger tout HTTP → HTTPS, cookies **Secure**
- Valider les certificats pour les appels serveur-à-serveur sensibles

Impact et CWE

- Vol massif d'identifiants / données personnelles
- Interception (man-in-the-middle)
- Non-conformité **RGPD, PCI-DSS**

CWE-327 Weak Crypto **CWE-319** Cleartext **CWE-916** Weak Hash
CWE-320 Key Mgmt

A05

Injection

Une donnée non fiable interprétée comme du code : SQL, HTML/JS, commandes système.

Dans cette catégorie

- La cause unique
- SQLi : le socle et ses limites
- XSS : encodage contextuel
- Injection de commande
- Impact et CWE

La cause unique : données mêlées au code



La parade est toujours la même

Séparer données et code : requêtes **paramétrées** (SQL), **encodage contextuel** en sortie (HTML/JS), **API sans shell** (commandes). La validation d'entrée vient **en plus**, jamais à la place.

Le socle : liaison de paramètres

✗ Vulnérable — concaténation

```
// la donnée devient de la syntaxe SQL
String q = "SELECT * FROM users "
    + "WHERE name = '" + name + "'";
stmt.executeQuery(q);
// name = "' OR 1=1 --" → toute la table
```

✓ Correctif — bind

```
// la valeur transite HORS de la chaîne SQL
jdbc.query(
    "SELECT id,email FROM users WHERE name = :n",
    Map.of("n", name), userMapper);
// PreparedStatement · :named · bind ORM
```

Pourquoi c'est robuste

La donnée est traitée comme une **valeur typée**, jamais comme de la syntaxe — l'apostrophe perd tout pouvoir. **Mais ce socle ne couvre pas tout** → diapo suivante.

Ce que la requête paramétrée ne protège PAS

Piège n°1 — les identifiants ne se lient pas

```
// X un nom de colonne / ORDER BY n'accepte pas '?'  
String q = "SELECT * FROM p ORDER BY "+col; // injectable  
// ✓ liste blanche : on ne concatène qu'une valeur SÛRE  
var COLS = Map.of("prix","prix","date","created_at");  
String safe = COLS.getOrDefault(col, "id");
```

ORM ≠ immunité : une requête **native** ou **HQL/JPQL concaténée** reste injectable. Le bind reste obligatoire.

La défense en profondeur, en vrai

- **Injection de second ordre** : donnée stockée « propre » puis réinjectée plus tard → paramétrer **partout**.
- **Moindre privilège** du compte SQL : pas de **DDL**, comptes lecture/écriture séparés, pas de **xp_cmdshell**.
- **Validation** (type, bornes) en complément.
- **Détection** : erreurs SQL journalisées, WAF, SAST/DAST en CI.

Encoder selon le contexte, pas « au hasard »

✗ Vulnérable — `th:utext` n'échappe rien

```
<div th:utext="${comment}"></div>
// comment = <script>vol_session()</script>
```

L'encodage dépend du CONTEXTE

```
Corps HTML → &lt; &gt; &amp; (th:text)
Attribut   → encodage attribut + guillemets
JavaScript → ne PAS injecter dans du JS
URL        → encodeURIComponent()
```

✓ Correctif — défense en profondeur

```
// 1) encodage contextuel automatique
<div th:text="${comment}"></div>
// 2) CSP stricte à base de nonce
Content-Security-Policy:
  script-src 'nonce-r4nd0m' 'strict-dynamic';
  object-src 'none'; base-uri 'none'
```

DOM XSS

L'encodage **serveur** ne protège pas des sinks JS (`innerHTML`), → `textContent` ,
Trusted Types, cookies `HttpOnly` .

Pas de shell, arguments séparés, entrée validée

X Vulnérable — le shell interprète

```
// concaténation dans un shell → RCE
Runtime.getRuntime().exec(
    "ping " + host);
// host = "8.8.8.8; rm -rf /"
// idem: sh -c, os.system(), eval()...
```

✓ Correctif — `ProcessBuilder` + allowlist

```
// 1) valider l'entrée (liste blanche stricte)
if (!host.matches("[a-zA-Z0-9.-]{1,253}"))
    throw new BadInput();
// 2) pas de shell : programme + args séparés
new ProcessBuilder("ping", "-c", "1", host)
    .redirectErrorStream(true).start();
// aucun métacaractère shell n'est interprété
```

Généralisation

Même logique pour toute injection : **LDAP** (échapper le DN), **NoSQL** (ne pas passer d'objets bruts), **XPath**, templates (**SSTI**). Le principe : ne jamais construire une commande par concaténation d'entrée.

Conséquences et contre-mesures

Impact

- Vol / altération de la base (SQLi)
- Vol de session, défiguration (XSS)
- Exécution de code serveur (commande → **RCE**)

CWE-89 SQLi **CWE-79** XSS **CWE-78** OS Command **CWE-943** Injection
générique

Contre-mesures

- **Requêtes paramétrées** / ORM (+ allowlist des identifiants)
- **Encodage en sortie** contextuel + CSP
- API **sans shell**, arguments séparés
- **Valider** toute entrée (liste blanche)

A01 → A05 (édition 2025) en un tableau

Catégorie	Le point dur	Correctif de référence
A01 Broken Access Control	Objet / fonction / champ (+ SSRF)	Refus par défaut, requêtes scopées, DTO, method security
A02 Security Misconfiguration	Défauts, surface, verbosité	Durcissement IaC, en-têtes, CORS restreint, erreurs génériques
A03 Supply Chain Failures	Dépendances et pipeline	Lockfile + hash, SCA/SBOM, signature (cosign), provenance (SLSA)
A04 Cryptographic Failures	Hash rapide, ECB, clé en dur	Argon2id, AES-GCM + KMS, TLS 1.3/HSTS
A05 Injection	Données mêlées au code	Bind + allowlist, encodage contextuel, ProcessBuilder

Rappel structure 2025

Security Misconfiguration passe #2, Supply Chain entre #3, Injection et Crypto reculent. **Insecure Design** devient A06 : il est traité en **partie 2**.

Les sigles du module en un coup d'œil

- OWASP** — Open Worldwide Application Security Project
- BOLA / IDOR** — Broken Object Level Auth. / accès objet
- SSRF** — Server-Side Request Forgery (→ A01 en 2025)
- SLSA** — cadre de provenance des artefacts
- SQLi / XSS** — injection SQL / Cross-Site Scripting
- CSP** — Content Security Policy (en-tête anti-XSS)
- IaC** — Infrastructure as Code

- CWE** — Common Weakness Enumeration (type de faiblesse)
- BFLA** — Broken Function Level Authorization
- SCA** — Software Composition Analysis (dépendances)
- KMS** — Key Management Service
- RCE** — Remote Code Execution
- HSTS** — HTTP Strict Transport Security
- OTP** — One-Time Password

- ASVS** — Application Security Verification Standard
- DTO** — Data Transfer Object (objet d'entrée dédié)
- SBOM** — Software Bill of Materials (inventaire)
- GCM** — Galois/Counter Mode (chiffrement authentifié)
- SSTI** — Server-Side Template Injection
- CORS** — Cross-Origin Resource Sharing

Trois réflexes transversaux



Ne jamais faire confiance à l'entrée

Valider, séparer donnée et code, DTO



Refus par défaut

Autorisation serveur, config minimale



Sécurité dès la conception

Threat modeling, patterns sûrs, primitives éprouvées

Et ensuite

La partie 2 traite **A06 → A10** (Insecure Design, Authentication Failures, Integrity Failures, Logging & Alerting, Mishandling of Exceptional Conditions) avec le même niveau de code.

Sources et références

Référentiels

- **OWASP Top 10:2025** — owasp.org/Top10
- **OWASP ASVS, Cheat Sheet Series, API Security Top 10**
- **CWE Top 25** (MITRE/CISA)
- **SLSA** — slsa.dev · **Sigstore** — sigstore.dev

Outils cités

- Spring Security, Argon2, Bucket4j, Google Tink
- Trivy, cosign, Renovate / Dependabot

Point de structure à valider

Ce support suit désormais l'édition **2025** (A02 = Misconfiguration, A03 = Supply Chain, A04 = Crypto, A05 = Injection). **Insecure Design**, qui était en A04 dans l'ancienne version 2021, devient **A06:2025** et bascule en **partie 2**. À harmoniser avec le module 6 lors de sa mise à jour.

À actualiser

Les paramètres cryptographiques recommandés et les versions d'outils évoluent ; le Top 10 est réédité tous les 3-4 ans (prochaine révision attendue vers 2028-2029).

Fin du module 5

Des questions ?

A01 à A05 (OWASP Top 10:2025) : du **code vulnérable** réaliste au **correctif de production**. Place à la seconde moitié du Top 10.